Capítulo 3

Gestión de Soluciones - III

Julio Alberto López Gómez

Este capítulo profundiza en dos soluciones de almacenamiento y gestión de datos propias de las bases de datos NoSQL: **las bases de datos XML y las bases de datos documentales.**

3.1. Bases de datos NoSQL

El término **NoSQL** proviene del inglés y está formado por las palabras "No" y "SQL". El significado literal del término no debe confundir al lector ni alejarlo de su correcto significado. NoSQL no engloba solamente un conjunto de bases de datos y sistemas de almacenamiento que no utilizan SQL para su gestión. El significado de NoSQL amplía lo anterior, siendo el acrónimo de ***"Not only SQL"***. Por tanto, NoSQL es una categoría general de sistemas de bases de datos que, además de SQL, utilizan otra serie de tecnologías para almacenar y gestionar los datos. El término NoSQL fue acuñado en 1998 por Carlo Strozzy y retomado en el año 2009 por Eric Evans, quien aprovechó este término para referirse a esta nueva generación de familias de bases de datos como **"Big Data".** (ver [Tiw11]).

Las transacciones realizadas en los **sistemas de bases de datos relacionales** garantizan la consistencia y escalabilidad de las operaciones a través de una serie de características especificadas en el **modelo ACID.**

- **Atomicidad (A):** Garantiza que una transacción se ha realizado completamente o no, evitando que algunas operaciones se ejecuten a medias.
- **Consistencia (C):** También conocida como **integridad**, asegura que cualquier transacción realizada desde un estado seguro en la base de datos conducirá a esta a otro estado también seguro o válido.

- **Aislamiento (I: Isolation):** Permite que el resultado en el estado del sistema sea el mismo, independientemente del modo de ejecución de las transacciones (secuencial o concurrente).
- **Durabilidad (D):** O **persistencia**, garantiza que a pesar de que se produzca un fallo en el sistema, los cambios realizados por una transacción persistirán.

Alternativamente, las bases de datos **NoSQL** siguen el modelo conocido como **BASE**, más flexible que ACID en aquellos sistemas de bases de datos que no se adhieren estrictamente al modelo relacional.

- **Disponibilidad básica (BAsic Availability):** El sistema siempre ofrece una respuesta a una petición de datos, aunque los datos sean inconsistentes, estén en fase de cambio o incluso se produzca un fallo.
- **Soft-state (S):** La consistencia de este tipo de bases de datos es eventual, por lo que el estado del sistema cambia a lo largo del tiempo, aunque no haya habido entradas de datos.
- **Eventual Consistency (E):** Cuando se dejan de recibir datos, el sistema se vuelve consistente. Así pues, los datos se propagan a todo el sistema, pero este continúa recibiendo datos sin evaluar la consistencia de los mismos para cada transacción antes de avanzar a la siguiente.

En la actualidad, las aplicaciones web modernas, el auge de la computación ubicua y el Big Data, presentan desafíos muy diferentes a los que presentan los sistemas de información tradicionales, implementados por medio de sistemas de bases de datos relacionales. Algunos **desafíos** son: el procesamiento masivo de datos, la alta frecuencia de lecturas y escrituras, los cambios dinámicos y frecuentes en el esquema de datos, la escalabilidad a costes razonables, gestión de datos temporales...En este contexto, en los últimos años han aparecido multitud de enfoques, modelos y tecnologías para aportar soluciones a los desafíos mencionados anteriormente. En este capítulo, se profundiza en **las bases de datos XML y las bases de datos documentales.**



Base de datos NoSQL: Sistema de bases de datos no relacional que utiliza distintas tecnologías para la gestión y almacenamiento distribuido de datos masivos sin un esquema estricto subyacente. Esta familia de base de datos permite incrementar la disponibilidad y escalabilidad del sistema, mejorando su rendimiento.

3.2. Bases de datos XML

El lenguaje **XML** (eXtensible Markup Language) es un lenguaje de marcas extensible derivado del lenguaje **SGML** (Standard Generalized Markup Language). Se trata de un lenguaje ideado para la definición y gestión de documentos que permite representar datos estructurados y semi-estructurados. En la actualidad, XML se ha convertido en un formato estándar para la comunicación entre aplicaciones, facilitando su integración. Entre las principales ventajas de XML destacan:

- **Datos autodocumentados:** Gracias a la posibilidad de definir cualquier tipo de etiquetas, no es necesario conocer el esquema para entender el significado de los datos.
- **Extensión:** Permite adaptar el lenguaje fácilmente a cualquier dominio.
- **Anidamiento:** La definición de estructuras anidadas dota a los documentos de gran flexibilidad para transferir y consultar información.
- **Flexibilidad:** Los documentos XML no tienen un formato estricto ni rígido, permitiendo añadir o ignorar información en ellos.

Estructura de un documento XML

La estructura de un documento XML está compuesta de elementos. Un **elemento** es una porción de documento delimitado por un par de etiquetas $<>$ (apertura) y $</>$ (cierre) entre las cuales se incluye un texto llamado **contexto** del elemento. Un elemento sin contexto puede abreviarse mediante el uso de una única etiquete $< nombre_elemento/ >$. La estructura de un XML se forma **anidando** elementos para especificar la estructura del documento, formando una **estructura de árbol**. El fragmento de código 3.1 representa la estructura básica de un documento XML que representa un listado de asignaturas.

Listado 3.1: Listado de Asignaturas

```
<Asignaturas_Primero>
    <Asignatura>
        <titulo> Cálculo </titulo>
        <tipo> Anual </tipo>
        <profesor>
            <nombre> Ana </nombre>
            <apellido> Sanz </apellido>
        </profesor>
        <profesor>
            <nombre> Roberto </nombre>
            <apellido> Hernández </apellido>
        </profesor>
    </Asignatura>
    <Asignatura>
        <titulo> Física </titulo>
        <tipo> Semestral </tipo>
        <profesor>
```

```
18              <nombre> Carmelo </nombre>
19              <apellido> Moya </apellido>
20          </profesor>
21      </Asignatura>
22  </Asignaturas_Primero>
```

Los elementos de un documento pueden contener atributos. Los **atributos** se incluyen en la definición del elemento, dentro de la etiqueta de apertura. Sintácticamente, se representan mediante un par $nombre = valor$. A diferencia de un subelemento, **un atributo solo puede aparecer una vez en una etiqueta.**
El uso de atributos y su diferencia con respecto a los elementos no siempre es evidente. A **nivel de documento**, los atributos se introducen para añadir texto que no se visualizará en el documento, mientras que los subelementos formarán parte del contenido del documento. A **nivel de datos**, la diferencia es insustancial, puesto que la misma información se puede representar utilizando atributos o subelementos. Por ejemplo, el subelemento $< tipo >$ de una asignatura, podría representarse como un atributo del elemento asignatura. El listado 3.2 representa la asignatura de Cálculo del ejemplo anterior, incluyendo el título de la asignatura como atributo de la misma.

Listado 3.2: Uso de atributos para la definición de asignaturas

```
1  <Asignaturas_Primero>
2      <Asignatura>
3          <titulo> Cálculo tipo = 'anual' </titulo>
4          <tipo> Anual </tipo>
5          <profesor>
6              <nombre> Ana </nombre>
7              <apellido> Sanz </apellido>
8          </profesor>
9          <profesor>
10             <nombre> Roberto </nombre>
11             <apellido> Hernández </apellido>
12         </profesor>
13 </Asignaturas_Primero>
```

Dado que una de las principales aplicaciones de XML es el intercambio de información entre organizaciones, y ante la flexibilidad del lenguaje que permite definir cualquier nombre para una etiqueta, es posible que **existan conflictos entre los nombres de éstas.** Para ello, se recurre a los **espacios de nombres**, los cuales permiten a las organizaciones especificar nombres de etiquetas únicos. De esta forma, se añade al nombre de la etiqueta o atributo un identificador de recursos universal. El listado muestra de forma resumida un ejemplo de utilización del espacio de nombres para la definición de asignaturas.

Listado 3.3: Uso de un espacio de nombres

```
1  <Asignatura xmlns:AS="http://www.listado-asignaturas.com">
2      <AS:titulo> Cálculo tipo = 'anual' </AS:titulo>
3      <AS:tipo> Anual </AS:tipo>
```

```
    <AS:profesor>
        <AS:nombre> Ana </AS:nombre>
        <AS:apellido> Sanz </AS:apellido>
    </AS:profesor>
    <AS:profesor>
        <AS:nombre> Roberto </AS:nombre>
        <AS:apellido> Hernández </AS:apellido>
    </AS:profesor>
</Asignatura>
```

Al igual que ocurre en las bases de datos relacionales, las bases de datos XML presentan una serie de **lenguajes de definición de tipos de documentos XML.** Concretamente, a continuación se profundizaran en **DTD y XML Schema.** Seguidamente, se estudiarán los lenguajes **XPath, XSLT y XQuery, como lenguajes de manipulación de datos XML** para la definición y ejecución de consultas.

3.2.1. Definition Type Document (DTD)

En XML, se dice que un documento está **bien formado** si cumple con la sintaxis del lenguaje: correcta definición de elementos y atributos, correcto anidamiento...Como se puede apreciar, este concepto alude a la **dimensión sintáctica de los documentos.**
Por otra parte, se dice que un documento XML **es válido** cuando **está bien formado y cumple con la estructura de documento que ha sido dada a través de un lenguaje de definición de datos.** Esta definición de la estructura de los documentos XML es **opcional**, si bien es cierto que es recomendable, especialmente cuando se trabaja con documentos complejos y con un volumen grande de ellos. La definición de tipos de documento o **DTD permite definir la estructura de un documento XML.** Para ello, la DTD especifica la estructura de los elementos y atributos de un documento: qué elementos pueden aparecer, qué atributos, qué subelementos, multiplicidad de ellos... Existen multitud de validadores on-line de documentos XML y sus correspondientes DTDs y/o XML Schemas[1].

La **declaración de una DTD** dentro de un documento XML puede implementarse de dos formas: en primer lugar, de manera **interna.** En este caso, se añade la DTD después del preámbulo XML y justo antes del documento propiamente dicho. El listado 3.4 muestra un ejemplo de declaración interna de una DTD.

Listado 3.4: Declaración Interna DTD

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<!DOCTYPE Elemento_Raiz [Aquí la DTD]>
  <!-------Aquí va el documento XML----->
```

[1] https://www.xmlvalidation.com
http://xmlvalidator.new-studio.org

En segundo lugar, la **DTD puede declararse de forma externa**, definiendo la misma en un fichero .DTD aparte que después es enlazado en el documento. El listado 3.5 muestra un ejemplo de declaración externa de una DTD.

Listado 3.5: Declaración Externa DTD

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2 <!DOCTYPE Elemento_Raiz SYSTEM "Mi_DTD.dtd">
3   <!-------Aquí va el documento XML----->
```

Siguiendo con el ejemplo del listado 3.1, el listado 3.6 muestra el archivo .dtd que validaría la estructura del documento de asignaturas.

Listado 3.6: DTD Listado de Asignaturas

```
1 <¡ DOCTYPE Listado_Asignaturas[
2 <! ELEMENT Asignaturas_Primero (Asignatura+)>
3 <! ELEMENT Asignatura (titulo, tipo, profesor+)>
4 <! ELEMENT titulo   (#PCDATA)>
5 <! ELEMENT tipo     (#PCDATA)>
6 <! ELEMENT profesor (nombre, apellido)>
7 <! ELEMENT nombre   (#PCDATA)>
8 <! ELEMENT apellido (#PCDATA)>
9 ]>
```

En el listado anterior, se define en primer lugar el tipo de documento "Listado_Asignaturas". A continuación, y especificados entre corchetes, se describen los elementos que contendrá el documento, comenzando por el elemento raíz "Asignaturas_Primero". Este elemento estará formado por uno o más elementos "Asignatura" (especificado mediante el operador +). Seguidamente, el elemento "Asignatura" viene dado por un conjunto de tres subelementos: título, tipo y profesor. Además, cada asignatura puede tener uno o más profesores. Los elementos título y tipo vienen dados por cadenas de caracteres (denotadas con la instrucción #PCDATA) mientras que el elemento profesor estará formado por dos subelementos llamados "nombre" y "apellido" ambos definidos mediante cadenas de caracteres.

La definición de atributos para un elemento se hace a continuación de la definición del mismo utilizando $<!ATTLIST>$. El listado 3.7 muestra la DTD anterior en caso de que el tipo de asignatura sea considerado un atributo.

Listado 3.7: DTD Listado de Asignaturas con atributos

```
1 <¡ DOCTYPE Listado_Asignaturas[
2 <! ELEMENT Asignaturas_Primero (Asignatura+)>
3 <! ELEMENT Asignatura (titulo, profesor+)>
4 <! ATTLIST tipo #CDATA REQUIRED>
5 <! ELEMENT titulo   (#PCDATA)>
6 <! ELEMENT tipo     (#PCDATA)>
7 <! ELEMENT profesor (nombre, apellido)>
8 <! ELEMENT nombre   (#PCDATA)>
```

```
 9 <! ELEMENT apellido (#PCDATA)>
10 ]>
```

Finalmente, la tabla 3.1 muestra un resumen de definición de elementos y atributos en una DTD.

Tabla 3.1: Descripción DTD de elementos y atributos XML

ELEMENTO			ATRIBUTO		
Tipos	#PCDATA	Cadena de caracteres	**Tipos**	CDATA	Cadena de caracteres
	EMPTY	Elemento sin contexto		ID	Identificador
	ANY	Cualquier tipo		IDREF	Referencia a un ID
Operadores	|	Disyunción		IDREFS	Referencia a múltiples ID.
	+	Uno o más	**Valores**	#REQUIRED	Obligatorio
	*	Cero o más		#IMPLIED	Opcional
	?	Cero o uno		#FIXED	Valor fijo
				#DEFAULT	Valor por defecto

3.2.2. XML Schema

Se trata de un **lenguaje de definición de la estructura de documentos XML más sofisticado que DTD.** Aunque es **más complejo, soluciona muchos de los inconvenientes de las DTD.** De hecho, XML Schema es un superconjunto del lenguaje DTD especificado mediante la sintaxis de XML. Entre otros aspectos, XML Schema soporta no solo el tipo cadena de caracteres (soportado por DTD), sino también tipos numéricos, tipos complejos como listas y además permite al usuario definir sus propios tipos así como extender tipos de datos complejos mediante un mecanismo similar a la herencia. Además, XML Schema soporta el concepto de espacios de nombre, permitiendo reutilizar elementos de un esquema en otros.

Un XML Schema se especifica en un documento aparte del documento XML con extensión **.xsd.** Dentro de este archivo, que sigue las reglas sintácticas de cualquier documento XML, se especifican las definiciones de cada elemento del esquema, qué atributos incluye así como sus tipos de datos válidos asociados tanto a los elementos como a los atributos. De esta forma, **XML Schema permite definir un esquema para validar documentos XML, de forma más sofisticada a como lo hace DTD.** El listado 3.8 muestra un XML Schema para el ejemplo del listado de asignaturas.

Listado 3.8: XML Schema: Listado de asignaturas

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified" xmlns:xs="
       http://www.w3.org/2001/XMLSchema">
3    <xs:element name="Asignaturas_Primero">
4      <xs:complexType>
5        <xs:sequence>
6          <xs:element maxOccurs="unbounded" name="Asignatura">
7            <xs:complexType>
8              <xs:sequence>
9                <xs:element name="titulo" type="xs:string" />
10               <xs:element name="tipo" type="xs:string" />
11               <xs:element maxOccurs="unbounded" name="profesor">
12                 <xs:complexType>
13                   <xs:sequence>
14                     <xs:element name="nombre" type="xs:string" />
15                     <xs:element name="apellido" type="xs:string" />
16                   </xs:sequence>
17                 </xs:complexType>
18               </xs:element>
19             </xs:sequence>
20           </xs:complexType>
21         </xs:element>
22       </xs:sequence>
23     </xs:complexType>
24   </xs:element>
25 </xs:schema>
```

El listado anterior comienza definiendo un XML Schema en la segunda línea de código. Cualquier XML Schema deberá incluir esta línea de código. El atributo $attributeFormDefault = \text{“}unqualified\text{”}$ indica que no es necesario utilizar el prefijo del espacio de nombres para definir los atributos del esquema, mientras que el atributo $elementFormDefault = \text{“}qualified\text{”}$ indica que no es necesario utilizar el prefijo del espacio de nombre para definir los elementos del esquema. A continuación, se define el elemento raíz “Asignaturas_Primero” el cual se trata de un tipo complejo, puesto que no se define con un tipo de datos básico como entero, carácter, etc. Este elemento está formado por una secuencia de elementos del tipo “Asignatura” (el número máximo de elementos Asignatura no está limitado ya que $maxOccurs = \text{“}unbounded\text{”}$. Los elementos Asignatura, a su vez, están formados de una secuencia de elementos “nombre”, “tipo” (ambos de tipo cadena de caracteres) y un conjunto de elementos de tipo “profesor” no acotados, el cual se compone a su vez de los elementos “nombre” y “apellido”, ambos de tipo cadena de caracteres.

XML Schema permite representar la cardinalidad de un elemento, definiendo el número mínimo y máximo de ocurrencias mediante los atributos $minOccurs$ y $maxOccurs$. Además, también es posible añadir restricciones de forma que se limiten los valores de un elemento o atributo, se permita elegir entre una lista de posibles...Finalmente, XML Schema dispone de los contenedores $sequence$ para definir una secuencia ordenada de subelementos, $chocie$ para definir la elección entre posibles grupos de elementos o elementos y all para definir un conjunto no

ordenado de elementos. Al igual que sucedía con DTD, **existen distintas herramientas on-line que no solo permiten validar un documento XML a partir de su esquema XSD, sino que también permiten generar el esquema XSD a partir del archivo XML**[2].

3.2.3. XPath

En cualquier sistema de bases de datos, además de los lenguajes de definición de datos, los **lenguajes de manipulación de datos** son esenciales para poder **realizar consultas** y recuperar datos, realizar actualizaciones, etc.

XPath es un lenguaje de manipulación de datos XML. La principal característica de este lenguaje es que **interpreta un documento XML como una estructura de datos de tipo árbol**, donde los nodos y sus respectivas hojas se corresponden con los elementos y subelementos del documento XML. De esta forma, y utilizando comandos que permiten recorrer el documento de forma análoga a los comandos que se utilizan en una terminal de comandos para recorrer el sistema de archivos de un computador, es posible realizar consultas. La tabla 3.2 muestra un listado de los principales operadores utilizados en XPath.

Tabla 3.2: XPath: Operadores básicos

Operador	Significado
/	Navegación entre nodos del documento. Selecciona los nodos del nivel inferior
//	Navegación entre nodos del documento. Selecciona los nodos del nivel inferior de cualquier nodo especificado a continuación
.	Selecciona el nodo actual
..	Selecciona el nodo padre o nodo de nivel superior
|	Expresa alternativas
@	Selecciona un atributo de un nodo
*	Selecciona todos los nodos
[]	Agrupa otros operadores

Con estos operadores, veamos cómo realizar algunas consultas de ejemplo sobre el fragmento XML que especifica un listado de asignaturas.

[2] `https://www.freeformatter.com` `https://www.liquid-technologies.com/online-xsd-validator`,`https://extendsclass.com/xml-schema-validator.html`



Consulta 1. Recuperar los nombres de todas las asignaturas

```
/Asignaturas_Primero/Asignatura/titulo/text()
```



Consulta 2. ¿Cuántas asignaturas son impartidas por más de un profesor?

```
/Asignaturas_Primero/Asignatura[count(profesor)>1]
```



Consulta 3. ¿Quiénes son los profesores que imparten la asignatura de física?

```
/Asignaturas_Primero/Asignatura[titulo="Fisica"]/profesor/nombre
```

Es posible utilizar XPath de forma on-line a través de visualizadores[3] o, por ejemplo, instalando el plug-in XPatherizer de Notepad++, en caso de utilizar este último editor de código XML.

3.2.4. XSLT

Otro lenguaje de manipulación de datos XML es el lenguaje XSLT. **El lenguaje XSLT (XML StylesSheet Transformations) proviene del lenguaje XSL (XML StylesSheet)**. Este último permite especificar las opciones de formato de un documento XML en un archivo separado, aislando así la especificación del contenido y la presentación de un archivo XML. Podría decirse que XSL es a XML lo que CSS es a HTML. **El lenguaje XSLT permite especificar las opciones e formato de un documento XML, pudiendo transformar este documento en otros formatos como HTML, PDF, etc**. Dada la potencia de este lenguaje, que es capaz de transformar un archivo XML en otro que también pudiera ser XML, **se utiliza frecuentemente como lenguaje de consulta**

Las transformaciones XSLT se definen por medio de **plantillas**, que permiten a su vez recuperar contenido del documento XML a través de la utilización de expresiones XPath. La principal característica de XSLT es que la aplicación de plantillas se realiza de forma recursiva, lo cual recibe el nombre de **recursividad estructural.** El listado 3.9 presenta un ejemplo de aplicación de una transformación XSLT por medio de una plantilla.

[3] `https://download.cnet.com/XPath-Visualizer/3000-7241_4-75804649.html`, `https://www.freeformatter.com/xpath-tester.html`

Listado 3.9: XSLT: Ejemplo de plantilla

```
1 <xsl: template match = /Asignaturas_Primero/Asignatura >
2 <titulo_asignatura>
3   <xsl:value-of select = titulo/>
4 </titulo_asignatura>
5 </xsl:template>
```

Tal y como se puede observar en el fragmento anterior, una transformación o consulta XSLT se define mediante dos órdenes básicas: $match$ que especifica una expresión XPath para seleccionar uno o más nodos y $value-of select$ que devuelve los valores especificados de los nodos que se han obtenido como resultado de la expresión XPath. Merece la pena destacar que **cualquier texto o etiqueta del archivo XSLT que no esté en el espacio de nombre se copia a la salida sin cambios.** Esto quiere decir, que el resultado del fragmento anterior no son únicamente los títulos de las asignaturas, sino estos mismos delimitados por una etiqueta de apertura y cierre llamada $< titulo_asignatura >$. De esta forma, a partir de un documento XML se ha generado otro con el resultado de la consulta. A modo de resumen, la tabla 3.3 define los principales constructores XSLT.

Tabla 3.3: Principales Constructores XSLT

Constructor XSLT	Significado
$< xsl : template\ match = "expresion\ XPath" >$	Selecciona nodos a devolver
$< xsl : value - of\ select = "valor" >$	Devuelve valores de los nodos seleccionados
$< /xsl : template >< xsl : template\ match = " * " / >$	Devuelve todos los nodos que no coinciden con alguna otra plantilla
$< xsl : attribute\ nombre = "nombre\ atributo" >$	Añade un atributo al elemento precedente
$< xsl : apply - templates/ >$	Aplica la recursividad estructural
$< xsl : keyname = "nombre\ clave" match =$ "$elementos\ aplicar" use = "valor\ clave" / >$	Define una clave
$< xsl : value - of\ select =$ "$key("nombre\ clave" "valor")$"	Devuelve los nodos que coincidan con el valor especificado. Especifica operaciones de combinación (JOINS)
$< xsl : sort\ select =$ "$elemento\ o\ atributo" / >$	Devuelve los nodos resultados, ordenados por un elemento o atributo

Para trabajar con XSLT, se aconseja la descarga del software XML Copy Editor (`https://xml-copy-editor.sourceforge.io`).

3.2.5. XQuery

Es un **lenguaje de consulta de propósito general sobre datos XML estandarizado por el consorcio W3C.** XQuery procede del lenguaje de programación Quilt y, por este motivo, toma prestadas muchas características de otros lenguajes como XPath o SQL. De hecho, la sintaxis de XQuery es mucho más similar a la de SQL, al contrario que ocurría con lenguajes como XPath o XSLT.

Para definir una consulta en XQuery se construye una **expresión, denominada "FLWR"(*flower*, flor en inglés).** A continuación, se describen cada una de las sentencias de las que se compone la expresión:

- **FOR:** Obtiene una serie de variables cuyos valores son el resultado de una expresión XPath. Es el equivalente a la cláusula **FROM** del lenguaje SQL.
- **LET:** Define y renombra expresiones complejas para ser utilizadas en el resto de la consulta, reduciendo la complejidad y aliviando la sintaxis. Es una cláusula **opcional.**
- **WHERE:** En esta cláusula, llamada igualmente en SQL, se especifican condiciones sobre los resultados obtenidos en la cláusula FOR.
- **RETURN:** Especifica y define el resultado que se va a obtener de la consulta. Se trata del equivalente a la cláusula **SELECT** en SQL.

Siguiendo la sintaxis anteriormente mencionada, el listado 3.10 muestra un ejemplo para obtener un listado con las asignaturas anuales.

Listado 3.10: XQuery: Ejemplo de consulta

```
1 for $x in /Asignaturas_Primero/Asignatura
2 let $nombre_asignatura:= $x/titulo
3 where $x/tipo='anual'
4 return <asignatura_anual> $nombre_asignatura </asignatura_anual>
```

En el listado anterior,la cláusula for permitirá recorrer todos los elementos asignatura. Por su parte, let define una expresión en la que se asigna a $nombre_asignatura$ el título de la asignatura que se está recorriendo en cada momento. La cláusula $where$ impone el criterio de que el tipo de asignatura sea anual y, finalmente, $result$ define como resultado un fragmento de código XML donde el nombre de la asignatura anual (según la expresión indicada en let) aparecerá como resultado encerrado entre dos etiquetas XML llamadas $asignatura_anual$. Tal y como se puede apreciar, XQuery también permite generar nuevos documetnos XML a partir de consultas.

Al igual que SQL, XQuery dispone de multitud de funciones para la consulta de datos. Así, es posible utilizar la función $distinct()$ para eliminar duplicados en los resultados así como funciones de agregación como $sum()$ o $count()$ además de poder especificar funciones definidas por el usuario. XQuery también permite

utilizar "joins" incluyendo en la cláusula where el operador de combinación al igual que en SQL. Por otra parte, si bien es cierto que XQuery no posee el operador *group by* de SQL, este puede implementarse mediante la creación de consultas anidadas, las cuales sí que son soportadas por el lenguaje.

3.3. Bases de datos documentales: MongoDB

Los sistemas de bases de datos documentales u orientados a documentos son **sistemas de bases de datos NoSQL que almacenan datos, los cuales se estructuran en forma de documentos.** En los últimos años, han proliferado una gran variedad de sistemas de este tipo como pueden ser **Cassandra**[4]**, CouchDB**[5]**, Riak**[6] **o MongoDB**[7]**, sobre el cual trata esta sección.**

El nombre **MongoDB** proviene de la palabra inglesa **"homongous"** que significa enorme. **MongoDB es, por tanto, un sistema de base de datos NoSQL, open source, orientado a documentos y escrito en lenguaje C++.** Este sistema de bases de datos está disponible no solo para múltiples plataformas y sistemas operativos (Windows, Linux, OS X) sino también como servicio empresarial en la nube y se puede integrar con otros servicios como, por ejemplo, Amazon Web Services (AWS).

3.3.1. MongoDB: Características y aplicaciones

Tal y como se ha presentado anteriormente, MongoDB es un sistema de bases de datos documental u orientado a documentos. Esta orientación hace que los datos se almacenen de forma estructurada en forma de documentos, los cuales se acoplan sin problema en los tipos de datos utilizados por los lenguajes de programación. Además, esta concepción hace que una base de datos MongoDB disponga de un esquema dinámico y fácilmente modificable.

En este apartado se pretende poner de relieve **las principales características de MongoDB** que hacen que sea en la actualidad uno de los sistemas de bases de datos NoSQL más utilizados tanto en el ámbito académico como en el profesional.

- **Alto rendimiento:** Gracias a la definición de los documentos y la creación de índices que hace que las lecturas y escrituras se realicen de forma más rápida.
- **Alta disponibilidad:** MongoDB dispone de servidores replicados con restablecimiento automático maestro.

[4] `https://cassandra.apache.org/_/index.html`
[5] `http://couchdb.apache.org`
[6] `https://riak.com`
[7] `https://www.mongodb.com/es`

- **Fácil escalabilidad:** Permitiendo distribuir colecciones de documentos entre diferentes máquinas de forma muy sencilla.
- **Indexación:** Similar al concepto de índice en una base de datos relacional, permite crear índices e índices secundarios para mejorar el rendimiento de las consultas.
- **Consultas ad-hoc:** Al igual que en una base de datos relacional, MongoDB da soporte a la búsqueda por campos, consultas de rangos, uso de expresiones regulares... A todo lo anterior, se añade la posibilidad de ejecutar y devolver una función JavaScript definida por el programador.
- **Replicación:** Siguiendo el modelo maestro-esclavo. El maestro puede ejecutar comandos de lectura y escritura mientras que el esclavo solo tiene acceso de lectura y la posibilidad de realizar copias de seguridad. En caso de que el maestro caiga, el esclavo puede elegir un nuevo maestro para mantener el servicio de replicación.
- **Balanceo de carga:** MongoDB es capaz de ejecutarse en múltiples servidores, pudiendo balancear la carga y/o duplicar los datos para mantener el correcto funcionamiento del sistema aunque se produzca un fallo hardware.
- **Almacenamiento de archivos:** Todo lo anterior, facilita que este sistema de base de datos pueda ser usado con un sistema de archivos GridFS con balanceo de carga y replicación.
- **Agregación:** Proporciona operadores de agregación y la posibilidad de utilizar funciones MapReduce para el procesamiento de datos por lotes.
- **Ejecución de Javscript del lado del servidor:** Es posible realizar consultas utilizando JavaScript, de forma que éstas son enviadas directamente a la base de datos para ser ejecutadas.

Todas estas características hacen que, en la actualidad, MongoDB sea uno de los principales sistemas de bases de datos NoSQL elegidos en multitud de aplicaciones como: **almacenamiento y registro de eventos, comercio electrónico, juegos, aplicaciones móviles, almacenes de datos, gestión de estadísticas en tiempo real y cualquier aplicación que requiera llevar a cabo analíticas sobre grandes volúmenes de datos.**

3.3.2. MongoDB: Conceptos básicos, utilidades y herramientas

Aunque MongoDB es un sistema de bases de datos NoSQL con todo lo que ello implica, también ofrece toda la funcionalidad de la que disponen las bases de datos relacionales. Sin embargo, **la estructura de una base de datos MongoDB difiere de la de una base de datos relacional.**

En un servidor MongoDB es posible crear tantas bases de datos como se desee. El concepto de **base de datos** es en este caso equivalente al de **base de datos** en los sistemas relacionales. Una vez creada, la base de datos estará compuesta por una o más **colecciones.** El término colección en el ámbito NoSQL es el equivalente al concepto de **tabla** en los sistemas relacionales. Cada colección, por tanto, estará formada por un conjunto de **documentos** (o incluso ninguno, en cuyo caso la colección estaría vacía). El concepto de documento en NoSQL se corresponde con el concepto de **registro** en una base de datos relacional. Un documento estará compuesto por una serie de **campos**, al igual que lo están los registros de una base de datos relacional. En el ámbito NoSQL, y más concretamente en MongoDB, cada documento viene dado por un archivo **JSON** (`http://www.json.org/`) en el cual, siguiendo una estructura clave-valor se especifican las características de cada documento. El listado 3.11 muestra un ejemplo de documento que define un barco.

Listado 3.11: Definición json de un documento barco

```
1 {
2     name:'USS Enterprise-D',
3     operator:'Starfleet',
4     type:'Explorer',
5     class:'Galaxy',
6     crew:750,
7     codes:[10,11,12]
8 }
```

Para la administración del sistema de bases de datos, MongoDB pone a disposición de los usuarios las siguientes utilidades:

- **mongo:** Se trata del shell interactivo de MongoDB que permite insertar, eliminar, actualizar datos y realizar consultas, además de replicar la información, apagar servidores y ejecutar código JavaScript.
- **mongostat:** Herramienta de línea de comandos que muestra las estadísticas de una instancia en ejecución de MongoDB
- **mongotop:** Herramienta de línea de comandos que muestra la cantidad de tiempo empleado en la lectura y escritura de datos por parte de la instancia en ejecución
- **mongosniff:** Herramienta de línea de comandos que permite hacer un rastreo del tráfico de la red que va desde y hacia MongoDB.
- **mongoimport/mongoexport:** Herramienta de línea de comandos para importar y exportar contenido en o desde .json, .csv o .tsv entre otros.
- **mongodump/mongorestore:** Herramienta de línea de comandos para la creación de una exportación binaria del contenido de la base de datos.

Aunque MongoDB tiene una interfaz administrativa accesible desde `http://localhost:28017/` (siempre que se haya iniciado con la opción $mongod\ --rest$, existen algunas herramientas gráficas para la administración y uso de este sistema de base de datos como **UMongo** (`https://github.com/agirbal/umongo`) , el cual

es una aplicación open source de sobremesa para navegar y administrar un clúster MongoDB o **Robomongo** (`https://robomongo.org`), que también es una herramienta gráfica open source que incluye una terminal de comandos completamente compatible con el shell de mongo.

3.3.3. MongoDB: Operaciones CRUD

Cualquier sistema de datos permite definir operaciones básicas como son la creación de tablas e inserción de registros, lectura de datos, actualización y eliminación de registros. Estas operaciones básicas se conocen con el nombre de **CRUD (Create, Read, Update, Delete).** A continuación, se muestran distintos ejemplos para ilustrar la implementación de estas operaciones en MongoDB.

Una vez iniciada una sesión del Shell de Mongo, es posible crear una base de datos utilizando el siguiente comando.

```
test>use DB_Ejemplo

switched to db DB_Ejemplo
```

Para crear una colección de documentos “Barco” dentro de la base de datos DB_Ejemplo se puede escribir.

```
DB_Ejemplo>db.createCollection(“ships”)

{ ok: 1 }
```

Para visualizar las bases de datos almacenadas en el servidor se puede utilizar el comando *show dbs* mientras que para obtener un listado de las colecciones de la base de datos sobre la que se está trabajando, es posible utilizar el comando *show collections*. Por otra parte, para insertar un documento dentro de la colección, como el mostrado en el listado 3.11, se ejecuta el siguiente comando.

```
db.ships.insert({name:’USS-Enterprise-D’,operator:’Starfleet’,type:’Explorer’,class:
’Galaxy’,crew:750,codes:[10,11,12]})
```

Para actualizar el nombre del barco “USS Promotheus” por “USS Something” es posible escribir el comando.

```
db.ships.update({name : ’USS Prometheus’}, {name : ’USS Something’})
```

Mientras que si se pretenden establecer o cambiar varios atributos de un mismo documento, como por ejemplo el operador y la clase, se puede utilizar el comando *update* de la siguiente forma.

```
db.ships.update({name : 'USS Something'}, {$set : {operator : 'Starfleet', class :
'Prometheus'}})
```

Finalmente, la eliminación de algún atributo de un documento también es una operación de actualización que puede realizarse de la siguiente forma.

```
db.ships.update({name : 'USS Something'}, {$unset : {operator : 1}})
```

La eliminación de documentos de una colección puede realizarse de forma directa o bien utilizando expresiones regulares. A continuación se muestran dos comandos para ilustrar ambas formas de eliminación. El segundo de ellos elimina aquellos documentos que comienzan por "a".

```
db.ships.remove({name : 'USS Prometheus'})

db.ships.remove({name:{$regex:'A*'}})
```

Para leer y mostrar documentos, se utiliza el comando *find()*. A continuación se muestra un ejemplo en el que, el primer comando muestra un documento al azar de los existentes, el segundo muestra todos los documentos y lo hace de forma indexada en lugar de como texto seguido, el tercero muestra solo los nombres de los barcos y el último, encuentra un documento cuyo nombre sea "USS Defiant".

```
db.ships.findOne()

db.ships.find().prettyPrint()

db.ships.find({}, {name:true})

db.ships.findOne({'name':'USS Defiant'})
```

3.3.4. Mongo DB: Consultas y Agregación

Al igual que cualquier lenguaje de manipulación de datos, MongoDB dispone de las herramientas y operadores necesarios para realizar consultas sobre los documentos almacenados en las colecciones. A continuación, se muestran ejemplos de uso de los principales operadores.

Los operadores relaciones mayor que, menor que, mayor o igual que y menor o igual que se corresponden, respectivamente, con los operadores $gt, $lt, $gte, $lte. Además, también es posible utilizar el operador $regex para recuperar elementos que cumplan una expresión regular. A continuación, se muestran dos consultas que recuperan aquellos barcos que permitan subir a bordo a más de cien pasajeros y otra consulta para recuperar aquellos que dejen subir tan solo a 100 pasajeros o menos.

```
db.ships.find({crew:{$gt:100}})

db.ships.find({crew:{$lte:100}})
```

El operador $exists permite devolver aquellos documentos para los que existe o no un determinado atributo. El siguiente comando permite encontrar aquellos barcos para los cuales existe el campo “class”.

```
db.ships.find({class:{$exists:true}})
```

Las funciones de agregación son especialmente útiles en los lenguajes de consulta y manipulación de datos. De esta forma, $sum permite agregar mediante la operación suma una serie de valores, $avg permite obtener la media, $mnin y $max encontrar el valor máximo y mínimo de un atributo para el conjunto de elementos, $push introduce en un array los resultados de la consulta que se ha realizado, $addToSet es similar al anterior solo que sin incluir valores duplicados y, por último, $first y $last permiten obtener el primer y último documento en una consulta. A continuación, se muestra un ejemplo del uso de cada uno de estos operadores de agregación.

```
db.ships.aggregate([{$group:{_id:"$operator",num_ships:{$sum:"$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", num_ships : {$avg : "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", num_ships : {$min : "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", classes : {$push: "$class"}}}])

db.ships.aggregate([{$group : {_id : "$operator", classes : {$addToSet :
"$class"}}}])

db.ships.aggregate([{$group:{_id:"$operator",last_class:{$last: "$class"}}}])
```

Finalmente, MongoDB pone a disposición de los usuarios multitud de funciones útiles en la realización de consultas. La tabla 3.4 muestra algunas de las más utilizadas.

Tabla 3.4: Funciones útiles en MongoDB

Función	Significado
$project	Cambia el conjunto de documentos modificando sus claves y valores
$match	Operación de filtrado para reducir el número de elementos recuperados
$group	Operador de agregación para agrupar resultados
$sort	Ordena los documentos
$skip	Recupera los documentos a partir de un numero especificado por el usuario
$limit	Limita los resultados de la consulta al valor pasado como parámetro a la función
$unwind	Utilizado como equivalente al join de SQL

Bibliografía

[AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.

[BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.

[BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms.* Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.

[dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.

[FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.

[Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.

[GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.

[HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

[IGG03] C. Imhoff, N. Galemmo, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.

[Inm02] William H. Inmon. *Building the Data Warehouse,3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.

[JVVL03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.

[KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.

[MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.

[MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.

[PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma,, 2006.

[RB02] Pablo Rodríguez Bocca. Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos. Interoperabilidad, Monografía, 2002.

[RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.

[SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.

[Tiw11] Shashank Tiwari. *Professional NoSQL*. Wrox Press Ltd., 2011.

[TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.

Big Data Aplicado

Conforme a contenidos del «Curso de Especialización en Inteligencia Artificial y Big Data»



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Capítulo 4

Gestión de Soluciones - IV

Julio Alberto López Gómez

En este capítulo se profundizará en las **bases de datos orientadas a grafos** como solución a problemas de *Big Data*. En la actualidad, cualquier problema puede ser modelizado por medio de grafos. Esta estructura matemática **versátil y escalable** proporciona un mayor rendimiento a los sistemas que utilizan este tipo de bases de datos. Para ilustrar todos los conceptos de este capítulo, se utilizará **Neo4j** como plataforma de bases de datos orientada a grafos.

4.1. Bases de datos orientadas a grafos

Una base de datos orientada a grafos es un tipo de base de datos **NoSQL** como lo son también, por ejemplo, las bases de datos documentales. Un sistema de gestión de bases de datos orientadas a grafos es aquel **define métodos y operaciones CRUD sobre un modelo de datos representado mediante un grafo.** Por norma general, este tipo de bases de datos se implementan para su uso junto con sistemas OLTP, por lo que están diseñadas en términos de relaciones de integridad y optimizadas para ofrecer un rendimiento operacional.

Aunque existen diferentes tipos de modelos de datos basados en grafos, cada sistema de base de datos elegirá uno de ellos para representar el contenido de la base de datos. Cualquiera de estos modelos está inspirado en **la estructura matemática de un grafo para modelizar un conjunto de datos.** Matemáticamente, un grafo es un conjunto de **nodos o vértices**, que representan entidades de un dominio, y un conjunto de **aristas o enlaces**, que representan las relaciones o interacciones que se producen entre las entidades. La figura 4.1 muestra un ejemplo de grafo que representa las relaciones de seguimiento entre distintos canales en YouTube.

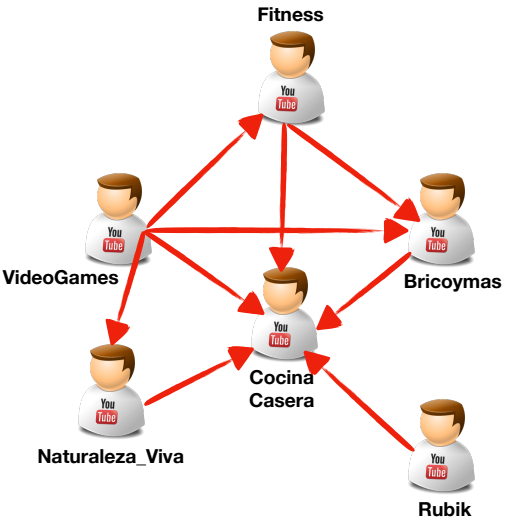


Figura 4.1: Ejemplo de grafo: relaciones de seguimiento entre canales YouTube

Uno de los modelos de datos basado en grafos más popular es **el modelo de grafo de propiedades etiquetadas**. Un grafo representado mediante este modelo, debe cumplir las siguientes características principales: **1)** debe contener un conjunto de nodos y aristas o relaciones entre dichos nodos, **2)** los nodos contienen propiedades, definidas a través de pares "clave-valor", **3)** los nodos pueden estar etiquetados con una o más etiquetas, **4)** las relaciones entre los nodos están nombradas y son dirigidas, teniendo siempre un nodo de inicio y otro nodo de fin, **5)** las relaciones del grafo también pueden contener propiedades. Este modelo, a pesar de su simplicidad, es sencillo de entender y permite modelar cualquier problema y/o conjunto de datos.

4.1.1. Características principales

El aumento del uso de los sistemas de bases de datos orientados a grafos y el auge que están alcanzando en los últimos años se debe, principalmente, a una serie de característica que confieren a estos sistemas una serie de ventajas importantes sobre los sistemas de bases de datos tradicionales y otros sistemas NoSQL.

- **Almacenamiento:** Las bases de datos orientadas a grafo pueden ser de almacenamiento nativo, en cuyo caso los datos se almacenan internamente con estructura de grafo. No obstante, existen bases de datos de grafos que mapean los datos para adptarlos a una estructura relacional.

- **Procesamiento:** El almacenamiento nativo permite obtener una estructura de datos que no requiere índices de adyacencia para referenciar los nodos del grafo, mejorando el rendimiento en la ejecución de consultas.
- **Rendimiento:** La ejecución de consultas sobre datos estructurados en forma de grafo proporciona escalabilidad al sistema de bases de datos, ya que aunque el número de datos aumente, las consultas se ejecutan sobre la porción del grafo de interés.
- **Flexibilidad:** Los modelos de datos basados en grafos permiten rápidamente añadir nuevos nodos, enlaces y subgrafos a un modelo existente con facilidad y sin pérdida de funcionalidad ni rendimiento.
- **Rapidez:** La naturaleza libre de esquema de las bases de datos orientadas a grafos, juntos con las APIs desarrolladas para su manipulación y los lenguajes de consulta, hacen que estos sistemas sean ágiles y rápidos, integrándolos con las metodologías de desarrollo de software iterativas e incrementales.

4.1.2. Áreas de Aplicación

Las características de las bases de datos orientadas a grafos desarrolladas en el apartado anterior confieren a esta tecnología un sinfín de **aplicaciones reales** en las que su uso mejora significativamente a las tecnologías tradicionales. A continuación, se muestran algunos ejemplos de aplicación donde, actualmente, este tipo de bases de datos es el más ampliamente utilizado.

- **Redes sociales:** El análisis de redes sociales es, a día de hoy, una de las principales fuentes de información de cualquier dominio. El análisis de redes sociales mediante bases de datos orientadas a grafos permite **identificar relaciones explícitas e implícitas** entre usuarios y grupos de usuarios, así como identificar la forma en la que ellos interactúan pudiendo inferir el comportamiento de un usuario en base a sus conexiones. En una red social, dos usuarios presentan una **conexión explícita** si están directamente conectados. Esto ocurre, por ejemplo, entre dos usuarios de Facebook que son amigos o dos compañeros de trabajo de la misma empresa en LinkedIn. Por otra parte, una **relación implícita** es aquella que se produce entre dos usuarios a través de un intermediario, como puede ser otro usuario, un post donde han hecho un comentario, un "like" o un artículo que ambos han comprado.
- **Sistemas de recomendación:** Estos sistemas permiten modelar en forma de grafo las relaciones que se establecen entre personas o usuarios y cosas, como pueden ser productos, servicios, contenido multimedia o cualquier otro concepto relevante en función del dominio de aplicación. **Las relaciones se establecen en función del comportamiento de los usuarios al comprar, consumir contenido, puntuarlo o evaluarlo etc.** De esta forma, los sistemas de recomendación **identifican recursos de interés** para un usuario específico y pueden **predecir su comportamiento al comprar un producto o contratar un servicio.**

- **Información geográfica:** Se trata del caso de aplicación más popular de la teoría de grafos. Las aplicaciones de las bases de datos orientadas a grafos en información geográfica van desde **calcular rutas óptimas entre dos puntos en cualquier tipo de red** (red de carreteras, de ferrocarril, aérea, logística...) **hasta encontrar todos los puntos de interés en un área concreta, encontrar el centro de una región u obtener la intersección entre dos o más regiones, entre otras muchas.** Así, las bases de datos orientadas a grafos permiten modelar estos casos de aplicación como grafos dirigidos sobre los cuales operar para obtener los resultados deseados.

Aunque en este apartado se hayan analizado las principales áreas de aplicación, son muchos los dominios de aplicación donde las bases de datos orientadas a grafos se están convirtiendo en una solución predominante, como lo son **la gestión de datos maestros, gestión de redes y centros de datos o control de acceso entre otros muchos.**

4.2. Neo4j

En esta sección se profundiza en una tecnología concreta de bases de datos orientadas a grafos. Aunque existe un amplio abanico de tecnologías que dan soporte a este tipo de bases de datos, como lo son **OrientDB**[1], **FlockDB**[2] o **InfiniteGraph**[3], en esta sección se profundiza en el trabajo con bases de datos orientadas a grafos utilizando **Neo4j**[4].

4.2.1. Introducción

Neo4j es una plataforma de bases de datos orientada a grafos lanzada en 2007 e implementada en Java. Entre sus características principales destacan **el almacenamiento y procesamiento de grafos nativo**, lo que la convierte en una alternativa potente para el trabajo en entornos reales, donde se requiere de estas características para mejorar el rendimiento y la escalabilidad de los sistemas. Además, a pesar de lo anterior, Neo4j también **permite trabajar con modelos de datos en forma de grafo de manera transaccional**, por lo que también es útil en sistemas transaccionales.

Junto a todo lo anterior, Neo4j dispone de **amplias librerías que permiten crear también modelos de aprendizaje automático sobre grafos.** Esta plataforma puede integrarse con múltiples lenguajes de programación como Python o C++, entre otros, y ofrece una serie de herramientas muy útiles para usuarios y desarrolladores. A continuación, se enumeran y detallan algunas de ellas:

[1] `https://orientdb.org`
[2] `https://github.com/twitter-archive/flockdb`
[3] `https://objectivity.com/infinitegraph/`
[4] `https://neo4j.com`

- **Graph Data Science Library:** Se trata de la librería principal que contiene una gran cantidad de algoritmos de búsqueda sobre grafos además de modelos de aprendizaje automático. Se trata de algoritmos y modelos eficientemente programados y escalables para el trabajo con grafos.
- **Neo4j Bloom:** Es una aplicación de visualización y exploración de grafos que permite la visualización de estos desde distintas perspectivas y puntos de vista, ofreciendo así una herramienta visual muy útil para visualizar los resultados de los algoritmos así como mostrar resultados a clientes.
- **Cypher:** Es el lenguaje de creación de consultas utilizado en Neo4j. Se trata de un lenguaje inspirado en SQL, solo que más sencillo y optimizado para el trabajo con grafos.
- **Integradores:** Con el objetivo de integrar y conectar Neo4j con otras plataformas y lenguajes, se han desarrollado distintos conectores para integrar esta plataforma con tecnologías como Apache Spark, Apache Kafka, BI...
- **Herramientas para desarrolladores:** Neo4j dispone de versiones para escritorio y web así como un *sandbox* en el que se proporciona un entorno controlado e integrado de los servicios Neo4j para desarrolladores.
- **Neo4j Aura:** Neo4j ofrece un servicio de computación en la nube para la creación de grafos y ejecución de algoritmos y modelos sobre ellos. Se trata de Neo4j Aura, el cual se encuentra disponible de forma gratuita y está integrado con Google Cloud Platform.

Las características anteriormente mencionadas y la cantidad de herramientas disponibles que Neo4j pone a disposición de los desarrolladores han sido los motivos principales para optar por esta plataforma para ilustrar el trabajo con bases de datos orientadas a grafos.

4.2.2. Importación de datos. Creación de grafos

Para comenzar a trabajar con Neo4j, es necesario **crear un grafo o bien partir de un grafo ya existente** que se importa dentro del área de trabajo de Neo4j. A continuación, se muestra cómo implementar ambos procedimientos.

Importación de un grafo

Sea el caso en el que se pretende importar un grafo a partir de un fichero **.csv.** En el ejemplo que se muestra a continuación, se dispone de dos ficheros .csv que contienen, respectivamente, **la definición de los nodos y de las aristas del grafo que se pretende importar.** Así pues, el listado 4.1 muestra los nodos del grafo. Para cada nodo se almacena su identificador, latitud, longitud y población.

Listado 4.1: Definición de nodos de un grafo

```
id,latitude,longitude,population
"Amsterdam",52.379189,4.899431,821752
"Utrecht",52.092876,5.104480,334176
"Den Haag",52.078663,4.288788,514861
"Immingham",53.61239,-0.22219,9642
"Doncaster",53.52285,-1.13116,302400
"Hoek van Holland",51.9775,4.13333,9382
"Felixstowe",51.96375,1.3511,23689
"Ipswich",52.05917,1.15545,133384
"Colchester",51.88921,0.90421,104390
"London",51.509865,-0.118092,8787892
"Rotterdam",51.9225,4.47917,623652
"Gouda",52.01667,4.70833,70939
```

Por su parte, el listado 4.2 especifica cada uno de los enlaces entre los nodos del grafo. Para cada uno de los enlaces se identifica el origen y destino del enlace, el tipo de relación que se define entre ellos y el coste.

Listado 4.2: Definición de aristas de un grafo

```
src,dst,relationship,cost
"Amsterdam","Utrecht","EROAD",46
"Amsterdam","Den Haag","EROAD",59
"Den Haag","Rotterdam","EROAD",26
"Amsterdam","Immingham","EROAD",369
"Immingham","Doncaster","EROAD",74
"Doncaster","London","EROAD",277
"Hoek van Holland","Den Haag","EROAD",27
"Felixstowe","Hoek van Holland","EROAD",207
"Ipswich","Felixstowe","EROAD",22
"Colchester","Ipswich","EROAD",32
"London","Colchester","EROAD",106
"Gouda","Rotterdam","EROAD",25
"Gouda","Utrecht","EROAD",35
"Den Haag","Gouda","EROAD",32
"Hoek van Holland","Rotterdam","EROAD",33
```

Dados estos dos ficheros, es posible **importar los nodos del grafo** según se indica en el listado 4.3.

Listado 4.3: Importación de nodos

```
WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/transport-nodes.csv"
      AS uri
LOAD CSV WITH HEADERS FROM uri AS row
MERGE (place:Place {id:row.id})
SET place.latitude = toFloat(row.latitude),
place.longitude = toFloat(row.latitude),
place.population = toInteger(row.population)
```

Así, en el listado anterior se define como identificador de recurso la dirección donde se encuentra el archivo que se pretende cargar, el cual es cargado a través de la instrucción **LOAD CSV.** La instrucción **MERGE** permite definir cada fila como nodo de un grafo de tipo lugar (*place*) y añadir a cada nodo la propiedad id cuyo valor será el identificador de la fila en la que se encuentra cada nodo dentro del fichero csv. Finalmente, los valores de latitud y longitud se convierten en valores de tipo real y la población en un valor de tipo entero.

Análogamente a la importación de nodos, el listado 4.4 muestra el código necesario para la **importación del fichero que contiene los enlaces** entre los nodos del grafo.

Listado 4.4: Importación de aristas

```
1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/transport-
      relationships.csv" AS uri
2 LOAD CSV WITH HEADERS FROM uri AS row
3 MATCH (origin:Place {id: row.src})
4 MATCH (destination:Place {id: row.dst})
5 MERGE (origin)-[:EROAD {distance: toInteger(row.cost)}]->(destination)
```

La diferencia principal de este listado con respecto al anterior, es que en este se necesita identificar mediante la instrucción **MATCH** los nodos de origen y destino de cada uno de los enlaces, lo cual es posible mediante los atributos “src” y “dst” incluidos en el archivo .csv. Finalmente, se utiliza la sentencia **MERGE** para crear una relación entre “origin” y “destination” de tipo **EROAD** que contiene la propiedad “distance” cuyo valor es el coste que aparecía en el fichero .csv.

Creación de un grafo

Otra alternativa es la **creación de un grafo desde cero**, especificando para ello sus nodos y enlaces. La figura 4.2 muestra un ejemplo de un **grafo de co-ocurrencia de hashtags** en el que cada uno de los nodos es un hashtag y los enlaces entre ellos se producen cuando dos hashtags aparecieron en un mismo tweet. Sobre cada enlace, se puede apreciar un valor que indica en cuántos tweets concurrieron los hashtags conectados por medio de dicha arista.

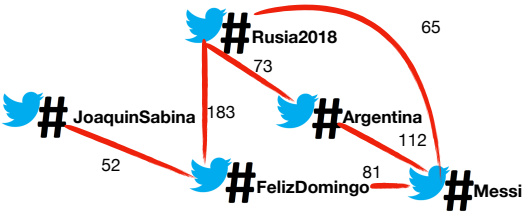


Figura 4.2: Grafo de co-ocurrencia de hashtags

Para representar este grafo en Neo4j, se puede escribir el siguiente fragmento de código que muestra el listado 4.5.

Listado 4.5: Creación grafo de co-ocurrencia de hashtags

```
CREATE
       (JS:Hashtag {name: 'JoaquinSabina'}),
       (RS:Hashtag {name: 'Rusia2018'}),
       (AG:Hashtag {name: 'Argentina'}),
       (FD:Hashtag {name: 'Feliz Domingo'}),
       (MS:Hashtag {name: 'Messi'}),

       (JS)-[:COOC {ntweet: 52}]->(FD),
       (FD)-[:COOC {ntweet: 52}]->(JS),
       (RS)-[:COOC {ntweet: 183}]->(FD),
       (FD)-[:COOC {ntweet: 183}]->(RS),
       (RS)-[:COOC {ntweet: 73}]->(AG),
       (AG)-[:COOC {ntweet: 73}]->(RS),
       (AG)-[:COOC {ntweet: 112}]->(MS),
       (MS)-[:COOC {ntweet: 112}]->(AG),
       (FD)-[:COOC {ntweet: 81}]->(MS),
       (MS)-[:COOC {ntweet: 81}]->(FD)
```

Tal y como se puede ver, en primer lugar se definen los nodos y sus propiedades y a continuación los enlaces y sus propiedades. **Todo grafo en Neo4j tiene aristas dirigidas, si bien es cierto que cuando se ejecutan las consultas puede no tenerse en cuenta la dirección de las aristas.** Por este motivo, se han especificado las aristas en ambas direcciones. En caso de querer crear una arista que sea interpretada como no dirigida, se debe crear sin el operador flecha que determina el sentido de la arista. Esto es posible hacerlo solo fuera de la instrucción **CREATE.** El listado 4.6 muestra cómo crear la primera relación del grafo solo que de forma no dirigida.

Listado 4.6: Creación de una arista no dirigida

```
MATCH(JS:Hashtag{name:'JoaquinSabina'})
MATCH(FD:Hashtag{name:'Feliz Domingo'})
MERGE (JS)-[:COOC{ntweet:52}]-(FD)
```

Para **mostrar el grafo**, es posible utilizar el siguiente comando. La figura 4.3 muestra cómo quedaría el grafo creado en el listado 4.5.

```
MATCH(n) return (n)
```

Para **eliminar un nodo** creado, es posible utilizar el comando:

```
MATCH(JS:Hashtag{name:'Joaquín Sabina'}) delete (JS)
```

Sin embargo, **un nodo solo puede ser eliminado si se eliminan las relaciones que contiene.** Para ello, es posible utilizar el comando:

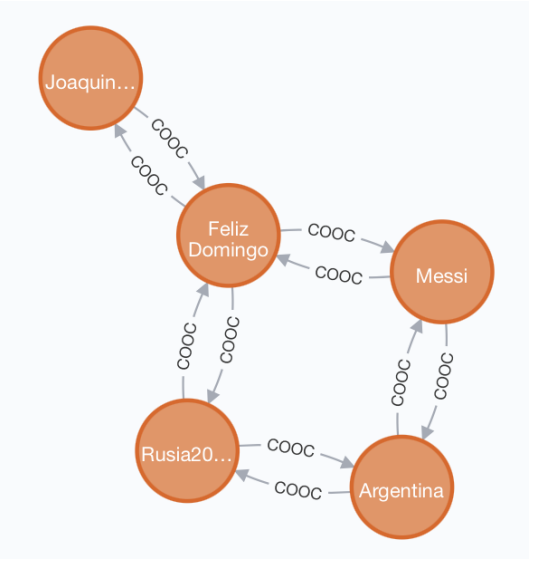


Figura 4.3: Visualización del grafo de co-ocurrencia de hashtags

```
MATCH(JS:Hashtag{name:'Joaquín Sabina'}) detach delete (JS)
```

Mientras que, **para eliminar todos los nodos y aristas,** es posible escribir:

```
MATCH(n) detach delete(n)
```

4.2.3. Recorridos sobre grafos

Los recorridos sobre grafos son una utilidad imprescindible en el trabajo con esta estructura de datos. Del mismo modo, los recorridos también son una herramienta fundamental cuando se trabaja con bases de datos orientadas a grafos.

Un recorrido sobre grafos es un procedimiento sistemático que permite explorar un grafo examinando todos sus vértices y aristas, comenzando desde un vértice inicial. A la hora de recorrer un grafo, existen dos tipos de recorridos: el **recorrido en anchura** (denotado BFS, por sus siglas en inglés) y el **recorrido en profundidad** (denotado DFS, también por sus siglas en inglés).

Recorrido en anchura (BFS)

El recorrido en anchura, también conocido por sus siglas en inglés *Breadth First Search* (BFS) es un procedimiento que permite recorrer un grafo, explorando todos sus nodos y aristas.

El recorrido en anchura parte de un nodo inicial. A continuación, el método comienza a explorar todos los nodos hijo del nodo inicial seleccionado que se encuentran a un salto. Una vez visitados, comienzan a explorarse los nodos hijo del siguiente nivel y así sucesivamente hasta haber recorrido el grafo completo. Este recorrido es muy utilizado cuando se busca un camino con el número mínimo de aristas entre dos vértices dados o cuando se busca un ciclo simple, es decir, una secuencia de nodos y aristas que se pueden describir circularmente en el que todos los nodos y aristas son distintos.

Por ejemplo, sea el grafo de la figura 4.4 que se corresponde con el grafo que se importó anteriormente. Es posible recorrerlo utilizando BFS escribiendo el siguiente código Neo4j que aparece en el listado 4.7.

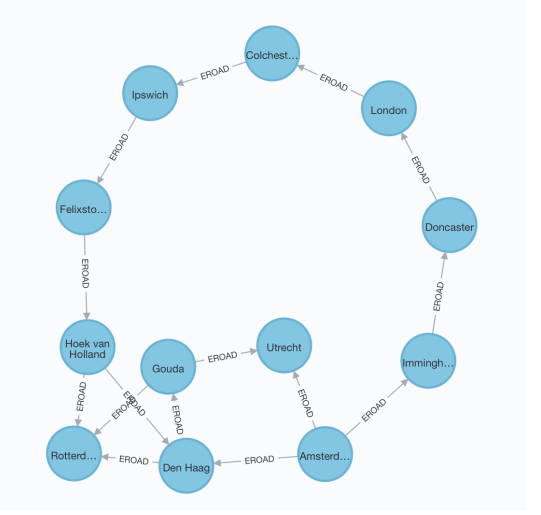


Figura 4.4: Visualización del grafo de ciudades

Listado 4.7: Recorrido en anchura

```
1 CALL gds.graph.create('myGraph','Place','EROAD',{relationshipProperties:'distance'})
2 MATCH (a:Place{id:'Doncaster'})
3 WITH id(a) AS startNode
4 CALL gds.alpha.bfs.stream('myGraph', {startNode: startNode})
5 YIELD path
6 UNWIND [ n in nodes(path) | n.id ] AS tags
7 RETURN tags
```

En el fragmento anterior, se define un grafo llamado *myGraph* el cual está formado por nodos de tipo *Place* y cuyas relaciones son del tipo **EROAD** y tienen una propiedad, denominada *distance*. A continuación, se define el nodo inicial con las sentencias **MATCH** y **WITH.** Después, se llama al método que realiza el recorrido BFS sobre el grafo anterior partiendo desde el nodo inicial para producir

un camino (*path*). Finalmente, se devuelven todos los nodos que forman parte del camino, mostrando los identificadores de cada uno de ellos. El resultado del recorrido es: Doncaster, London, Colchester, Ipswich, Felixstowe, Hoek van Holand, Den Haag, Rotterdam, Gouda, Utrecht.

Recorrido en profundidad (DFS)

El recorrido en profundidad, también conocido por sus siglas en inglés *Depth First Search* (DFS) es un procedimiento alternativo al recorrido en anchura para explorar un grafo, recorriendo todos sus nodos y aristas.

El procedimiento para recorrer un grafo mediante el método DFS es el siguiente: en primer lugar, se parte de un nodo inicial. A partir de él, se empiezan a recorrer todos sus hijos hasta el último nivel. Una vez llegados al último nivel del grafo, se vuelve hacia atrás recorriendo todos los hijos de los nodos de niveles anteriores.

En Neo4j, el procedimiento para recorrer un grafo utilizando el recorrido BFS se muestra en el listado de código 4.8. El procedimiento, como se puede comprobar, es prácticamente idéntico al del listado 4.7 solo que cambiando el método de recorrido. El resultado obtenido es: Doncaster, London, Colchester, Ipswich, Felixstowe, Hoek van Holland, Rotterdam, Den Haag, Gouda, Utrecht.

Listado 4.8: Recorrido en profundidad

```
1 CALL gds.graph.create('myGraph','Place','EROAD',{relationshipProperties:'distance'})
2 MATCH (a:Place{id:'Doncaster'})
3 WITH id(a) AS startNode
4 CALL gds.alpha.dfs.stream('myGraph', {startNode: startNode})
5 YIELD path
6 UNWIND [ n in nodes(path) | n.id ] AS tags
7 RETURN tags
```

4.2.4. Caminos mínimos

El cálculo y la obtención de caminos mínimos dentro de un grafo es uno de los problemas clásicos en teoría de grafos con multitud de aplicaciones en el mundo real. **Un camino dentro de un grafo es un conjunto de nodos y aristas que conectan un par de nodos del grafo.** En el caso de los **grafos no pesados**, es decir, aquellos cuyas aristas no tienen coste, el cálculo del camino mínimo se obtiene **minimizando el número de saltos** entre el nodo origen y el nodo destino. En el caso de los **grafos pesados**, aquellos cuyas aristas tienen coste, el cálculo del camino mínimo se obtiene a través del camino en el que **la suma de los costes de sus aristas es mínima.**

Junto con los recorridos de grafos, los métodos de obtención de caminos mínimos son muy útiles cuando se trabaja en **entornos dinámicos**, donde aparecen y desaparecen continuamente aristas del grafo o sus costes cambian dinámicamente. El cálculo de caminos mínimos también es muy útil en aplicaciones que deben dar

respuestas en tiempo real. Algunos ejemplos claros pueden ser encontrar rutas entre dos localidades, como ocurre en Google Maps u obtener los grados de separación entre una empresa y un empleado potencial al que se quiere contratar, como puede ocurrir en LinkedIn.

El **algoritmo de Dijkstra** es uno de los algoritmos más utilizados en teoría de grafos para la obtención de caminos mínimos. Dado el grafo de ciudades con el que se está trabajando a lo largo de este capítulo, el listado 4.9 permite obtener **los caminos mínimos entre un nodo del grafo y todos los demás nodos alcanzables.**

Listado 4.9: Ejecución del algoritmo de Dijkstra sobre el nodo Amsterdam

```
MATCH (source:Place{id:'Amsterdam'})
CALL gds.allShortestPaths.dijkstra.stream('myGraph', {
    sourceNode: source,
    relationshipWeightProperty: 'distance'
})
YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
RETURN
    index,
    gds.util.asNode(sourceNode).id AS sourceNodeName,
    gds.util.asNode(targetNode).id AS targetNodeName,
    totalCost,
    [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
    costs,
    nodes(path) as path
ORDER BY index
```

En el listado anterior, se utiliza la sentencia **MATCH** para definir el nodo de origen y, posteriormente, se guarda el grafo incluyendo el nodo de origen y la propiedad de distancia que es la utilizada para obtener el camino mínimo. A continuación, se producirá un conjunto de caminos en el que se mostrará información completa del nodo de origen, destino, coste total y camino recorrido así como los distintos costes parciales. En caso de querer obtener el camino mínimo entre un par de nodos concretos, es posible utilizar el código mostrado en el listado 4.10 análogo al listado 4.9.

Listado 4.10: Ejecución del algoritmo de Dijkstra sobre el par de nodos Amsterdam-London

```
MATCH (source:Place{id:'Amsterdam'}), (target:Place{id:'London'})
CALL gds.shortestPath.dijkstra.stream('myGraph', {
    sourceNode: source,
    targetNode: target,
    relationshipWeightProperty: 'distance'
})
YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
RETURN
    index,
    gds.util.asNode(sourceNode).name AS sourceNodeName,
    gds.util.asNode(targetNode).name AS targetNodeName,
    totalCost,
    [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
    costs,
```

```
    nodes(path) as path
ORDER BY index
```

El camino mínimo entre Amsterdam y Londres es el formado por: Amsterdam Immingham, Doncaster, London. Este camino tiene un coste total de 720.

El **método A* es una técnica de búsqueda informada** que permite, al igual que el algoritmo de Dijkstra, **calcular el camino mínimo entre dos nodos dados cualesquiera.** Al contrario que el algoritmo de Dijkstra, cuando se va a seleccionar el siguiente nodo del camino, la técnica A* no solo se tiene en cuanta la distancia ya calculada, sino que **este algoritmo combina la distancia calculada con el resultado de una función heurística.** Esta función toma un nodo como entrada y devuelve un valor que corresponde con el coste de llegar al nodo objetivo desde ese nodo. Esta técnica, que se conoce también como de **búsqueda informada,** continúa el recorrido del grafo en cada iteración desde el nodo con el menor coste combinado entre la distancia ya calculada y la función heurística. El listado 4.11 muestra cómo obtener el camino mínimo mediante el método A* entre Amsterdam y Londres.

Listado 4.11: Ejecución del algoritmo A* sobre el par de nodos Amsterdam-London

```
CALL gds.graph.create(
    'myGraph2',
    'Place',
    'EROAD',
    {
        nodeProperties: ['latitude', 'longitude'],
        relationshipProperties: 'distance'
    }
)
MATCH (source:Place{id:'Amsterdam'}), (target:Place{id:'London'})
CALL gds.shortestPath.astar.stream('myGraph2', {
    sourceNode:source,
    targetNode:target,
    latitudeProperty:'latitude',
    longitudeProperty:'longitude',
    relationshipWeightProperty: 'distance'
})
YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
RETURN
    index,
    gds.util.asNode(sourceNode).id AS sourceNodeName,
    gds.util.asNode(targetNode).id AS targetNodeName,
    totalCost,
    [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
    costs,
    nodes(path) as path
ORDER BY index
```

Como se puede ver, este fragmento de código es similar a los vistos anteriormente, solo que a la hora de guardar el grafo, se almacenan los propiedades de latitud y longitud, necesarias en Neo4j para ejecutar el método A*.

4.2.5. Medidas de centralidad

La centralidad se define como la relevancia de un nodo dentro de un grafo. Por tanto, el estudio de medidas de centralidad permite **identificar nodos relevantes dentro de un grafo.** Esta identificación permitirá entender el comportamiento de la red, qué nodos permiten viralizar con mayor rapidez el contenido de la red, desde qué nodos la información está más accesible, etc. El estudio de medidas de centralidad tiene multitud de aplicaciones, aunque son especialmente notables las relacionadas con los ámbitos de la **publicidad y el marketing,** donde el estudio de estas métricas permite identificar actores relevantes dentro de una red a los que se puede contratar para anunciar un producto o identificar sobre qué actores enviar información para alcanzar a más usuarios.

Respecto a las medidas de centralidad, existe una única medida sino que, en función de los datos y del propósito que se pretende alcanzar, se utilizan unas métricas u otras. a continuación, se van a definir tres medidas de centralidad con las que se realizarán ejemplos en Neo4j: **centralidad de grado, cercanía e intermediación.**

Para trabajar con medidas de centralidad, se va a importar un grafo social que servirá de ejemplo para el cálculo de estas métricas. El listado 4.12 muestra el código necesario para su creación. Por otra parte, la figura 4.5 muestra una visualización del grafo importado.

Listado 4.12: Importación de un grafo social

```
1  WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/" AS base
2  WITH base + "social-nodes.csv" AS uri
3  LOAD CSV WITH HEADERS FROM uri AS row
4  MERGE (:User {id: row.id})
5
6  WITH base + "social-relationships.csv" AS uri
7  LOAD CSV WITH HEADERS FROM uri AS row
8  MATCH (source:User {id: row.src})
9  MATCH (destination:User {id: row.dst})
10 MERGE (source)-[:FOLLOWS]->(destination)
```

Centralidad de grado

El grado de un nodo en un grafo se define como el número de aristas o conexiones que entran o salen de un nodo. Así pues, en un nodo de un grafo se distinguen dos grados: el **grado de entrada**, correspondiente al conjunto de conexiones que entran a un nodo y el **grado de salida**, que se corresponde con el conjunto de aristas que salen de un nodo. El primero indica la **prominencia** de un nodo dentro de la red, mientras que el segundo la **influencia** del nodo dentro del grafo. Esta medida de centralidad es muy utilizada para identificar actores prominentes o influyentes o para identificar conductas fraudulentas, caracterizadas en ocasiones por una actividad anormal. Para calcular la centralidad de grado en este

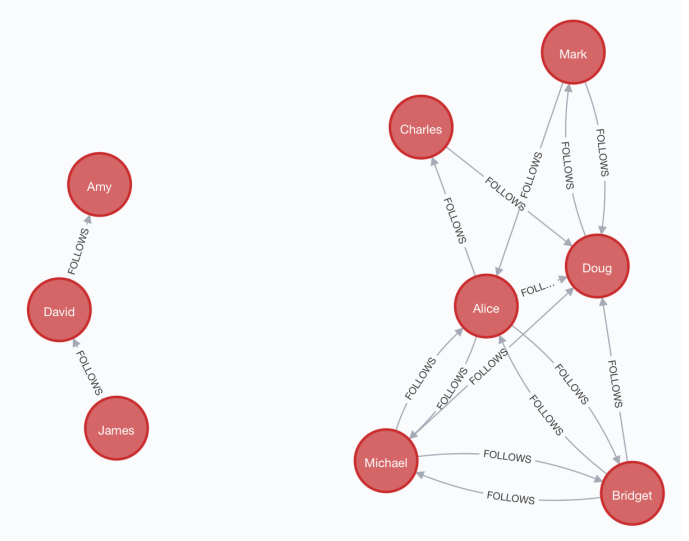


Figura 4.5: Visualización de un grafo social

grafo, se almacenará en primer lugar el grafo creado y, a continuación, se ejecutará el método que calcula la centralidad de grado según el código mostrado en el listado 4.13. El nodo con mayor grado es Doug, que tiene un grado de 5, seguido de Alice que tiene un grado de 3.

Listado 4.13: Cálculo de la centralidad de grado

```
CALL gds.graph.create(
  'myGraph',
  'User',
  {
    FOLLOWS: {
      orientation: 'REVERSE'
    }
  }
)
CALL gds.degree.stream('myGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS name, score AS followers
ORDER BY followers DESC, name DESC
```

Cercanía

La cercanía es una medida de centralidad que determina qué nodos del grafo expanden rápida y eficientemente la información a través del grafo. Para calcular esta medida, se obtiene **la suma del inverso de las distancias de un nodo al resto.** De esta forma, dado un nodo u la cercanía del mismo se calcula por medio de la ecuación (4.1).

$$C(u) = \frac{1}{\sum_{v=1}^{n-1} d(u,v)} \tag{4.1}$$

Donde n es el número de nodos del grafo y $d(u,v)$ es la distancia del camino mínimo entre u y v. **La cercanía es una métrica muy utilizada para estimar tiempos de llegada en redes logísticas, para descubrir actores en posiciones privilegiadas en redes sociales o para estudiar la prominencia de palabras en un documento en el campo de la minería de textos.** Para obtener la cercanía en el grafo social de ejemplo, se puede utilizar el fragmento de código mostrado en el listado 4.14. En el grafo de la imagen, Alice, Doug y David tienen una cercanía de 1.

Listado 4.14: Cálculo de la cercanía

```
CALL gds.alpha.closeness.stream({
  nodeProjection: User',
  relationshipProjection: FOLLOWS'
})
YIELD nodeId, centrality
RETURN gds.util.asNode(nodeId).name AS user, centrality
ORDER BY centrality DESC
```

Intermediación

La intermediación es una medida de centralidad que permite detectar la influencia que tiene un nodo o actor del grafo en el flujo de información o de recursos de la red. El cálculo de la intermediación permite identificar a nodos que hacen de puentes entre distintas porciones del grafo. Esta medida de centralidad **es muy utilizada para la identificación de influencers y el estudio de la viralización de mensajes en redes sociales.** Intuitivamente, la intermediación de un nodo **será mayor en tanto en cuanto dicho nodo aparezca en los caminos mínimos de cualquier otro par de nodos.** Más formalmente, la intermediación puede calcularse según la ecuación (4.2).

$$B(u) = \sum_{s \neq u \neq t} \frac{p(u)}{p} \tag{4.2}$$

Donde u es el nodo del cual se calcula la intermediación (B),s y t son nodos del grafo, $p(u)$ es el número de caminos mínimos entre s y t que pasan por u y p es el número de caminos mínimos entre s y t. Para el cálculo de la intermediación, se puede utilizar el fragmento de código mostrado en el listado 4.15. Al ejecutar este listado, se obtiene que Alice tiene la mayor intermediación, teniendo un valor de 10.

Listado 4.15: Cálculo de la intermediación

```
CALL gds.betweenness.stream('myGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC
```

4.2.6. Detección de comunidades

A la hora de trabajar con grafos reales, que presentan una gran cantidad de nodos y enlaces, muchas veces se pretende **identificar comunidades** dentro del grafo para aplicar algoritmos sobre ellas. **Una comunidad es, por tanto, un conjunto de nodos que presentan más relaciones entre sí que con el resto de nodos fuera de la comunidad.** La detección e identificación de comunidades permite **identificar comportamientos emergentes y de rebaño dentro de una red.** De esta forma, es posible detectar y predecir hábitos de los usuarios. A continuación, se van a aplicar tres métodos de detección de comunidades sobre el grafo social de ejemplo que se ha utilizado en secciones anteriores.

Conteo de triángulos

Un triángulo es un conjunto de tres nodos que tienen relaciones entre sí. El conteo de triángulos para un nodo dado, permite estudiar o inspeccionar de forma global un grafo y, aplicado sobre componentes conexas, permite inspeccionar regiones de un grafo. Para aplicar este método, es necesario almacenar el grafo teniendo en cuenta que las aristas deben almacenarse como **UNDIRECTED.** El listado 4.16 muestra el fragmento de código necesario para aplicar este método sobre el grafo social de ejemplo. La ejecución de este método da como resultado que Alice y Doug son aquellos que pertenecen a más triángulos, un total de 5.

Listado 4.16: Ejecución del conteo de triángulos

```
CALL gds.graph.create(
  'myGraph2',
  'User',
  {
    FOLLOWS: {
      orientation: 'UNDIRECTED'
    }
  }
```

```
)
CALL gds.triangleCount.stream('myGraph)
YIELD nodeId, triangleCount
RETURN gds.util.asNode(nodeId).name
AS name, triangleCount
ORDER BY triangleCount DESC
```

Coeficiente local de clustering

Este coeficiente **proporciona una medida cuantitativa del grado de agrupación de un nodo.** Para el cálculo del coeficiente local de clustering se utiliza el conteo de triángulos, de forma que se compara el grado de agrupación de un nodo con el grado máximo de agrupación que podría tener. Así pues, el coeficiente local de clustering se puede calcular mediante la ecuación (4.3).

$$CC(u) = \frac{2R_u}{k_u(k_u - 1} \tag{4.3}$$

Donde u es un nodo, $R(u)$ es el número de relaciones que a través de los vecinos de u (lo cual puede medirse con el número de triángulos que pasan por u) y $k(u)$ es el grado de u. El cálculo del coeficiente local de clustering puede realizarse por medio del listado 4.17. Los actores Bridget, Charles, Mark y Michael tienen el mayor coeficiente local de clustering, que es 1.

Listado 4.17: Cálculo del coeficiente local de clustering

```
CALL gds.localClusteringCoefficient.stream('myGraph2')
YIELD nodeId, localClusteringCoefficient
RETURN gds.util.asNode(nodeId).id
AS name, localClusteringCoefficient
ORDER BY localClusteringCoefficient DESC
```

Por último, **el coeficiente global de clustering se calcula como la suma normalizada de los coeficientes de clustering locales.** Así, estos coeficientes nos permiten encontrar medidas cuantitativas para detectar comunidades, pudiendo especificar incluso un umbral para establecer la comunidad (por ejemplo, especificar que los nodos han de estar conectados en un 40 %).

Componentes fuertemente conexas

En un grafo dirigido, una componente fuertemente conexa es aquel grupo de nodos en el que cualquier nodo puede ser alcanzado por cualquier otro en ambas direcciones. El estudio de las componentes fuertemente conexas en un grafo permite **estudiar la conectividad de la red.** Para realizar el cálculo de las

componentes conexas, el cual se realiza en un tiempo de procesamiento proporcional al número de nodos, es posible utilizar el listado 4.18. El resultado permite comprobar que hay un total de cuatro componentes fuertemente conexas en el grafo social.

Listado 4.18: Obtención de componentes fuertemente conexas

```
CALL gds.alpha.scc.stream({
  nodeProjection: 'User',
  relationshipProjection: 'FOLLOWS'
})
YIELD nodeId, componentId
RETURN gds.util.asNode(nodeId).id AS Name, componentId AS Component
ORDER BY Component DESC
```

4.2.7. Predicción de enlaces

Los grafos son estructuras de datos que representan **sistemas dinámicos**, que evolucionan a lo largo del tiempo. Por este motivo, es muy común que en un grafo cualquiera aparezcan y desaparezcan nuevos nodos y conexiones entre dichos nodos. **Los métodos de predicción de enlaces permiten predecir qué enlaces se formarán próximamente entre los nodos del grafo, permitiendo adelantarse a los acontecimientos y prevenir eventualidades.** Como norma general, los métodos de predicción de enlaces **se basan en medidas de cercanía y de centralidad** asumiendo que los nuevos enlaces se producirán, mayoritariamente, sobre/desde los nodos más relevantes. A continuación, se detalla el funcionamiento de tres métodos comúnmente utilizados en predicción de enlaces.

Vecinos comunes

El método de vecinos comunes se basa en la idea genérica de que dos actores de la red que tienen una relación con un usuario común tendrán más posibilidad de conectarse entre sí que quienes no. Formalmente, dados dos nodos, la posibilidad de que se produzca un enlace entre los nodos x e y viene dada por la ecuación 4.4.

$$CN(x, y) = |N(x) \cap N(y)| \tag{4.4}$$

Donde $N(x)$ es el conjunto de nodos adyacentes a x y $N(y)$ es el conjunto de nodos adyacentes a y. Cuanto mayor es el valor de CN calculado, existe mayor posibilidad de que se produzca un nuevo enlace entre x e y. En el grafo social de ejemplo, el cálculo de vecinos comunes para Charles y Bridget se puede realizar a través del listado 4.19. El resultado de este cálculo es 2.

Listado 4.19: Obtención del valor de vecinos comunes para Charles y Bridget

```
MATCH (x:User{id:'Charles'})
```

```
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.commonNeighbors(x,y) AS score
```

Adhesión preferencial

Este método se basa en la idea general de que cuanto más conectado está un nodo, es más probable que reciba nuevos enlaces. Formalmente, esta idea se puede expresar por medio de la ecuación 4.5

$$CN(x, y) = |N(x) \cdot N(y)| \tag{4.5}$$

En el grafo social de ejemplo, el cálculo de adhesión preferencial para Charles y Bridget se puede realizar a través del listado 4.20. El resultado de este cálculo es 10.

Listado 4.20: Obtención del valor de adhesión preferencial para Charles y Bridget

```
1 MATCH (x:User{id:'Charles'})
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.preferentialAttachment(x, y) AS score
```

Asignación de recursos

Se trata de una métrica compleja que evalúa la cercanía de un par de nodos para determinar la posibilidad de que, entre ellos, se produzca un nuevo enlace. Para ello, se utiliza la expresión dada en la ecuación 4.6.

$$RA(x, y) = \sum_{U \in N(x) \cap N(y)} \frac{1}{N(u)} \tag{4.6}$$

En el grafo social de ejemplo, el cálculo de la asignación de recursos para Charles y Bridget se puede realizar a través del listado 4.21. El resultado de este cálculo es 0.309.

Listado 4.21: Obtención del valor de asignación de recursos para Charles y Bridget

```
1 MATCH (x:User{id:'Charles'})
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.resourceAllocation(x, y) AS score
```

Bibliografía

[AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.

[BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.

[BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms.* Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.

[dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.

[FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.

[Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.

[GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.

[HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

[IGG03] C. Imhoff, N. Galemmo, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.

[Inm02] William H. Inmon. *Building the Data Warehouse,3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.

[JVVL03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.

[KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.

[MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.

[MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.

[PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma,, 2006.

[RB02] Pablo Rodríguez Bocca. Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos. Interoperabilidad, Monografía, 2002.

[RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.

[SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.

[Tiw11] Shashank Tiwari. *Professional NoSQL*. Wrox Press Ltd., 2011.

[TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.